

6th place solution: SuperPoint + LightGlue

Rank	6
Team	Pavel Kazlou
Author	Pavel Kazlou
Topic ID	700586
Version	Original

Source: <https://www.kaggle.com/competitions/recodai-luc-scientific-image-forgery-detection/discussion/700586>
Retrieved with Kaggle CLI on 2026-07-18.

- Public LB: 0.322 (398th)
- Private LB: 0.336 (6th)
- [Submission notebook](#)

Overview

The pipeline for detecting matching regions was pretty simple:

1. **Tile Detection:** segmenting the input image into rectangular tiles to be compared in pairs.
2. **Feature Extraction:** using SuperPoint (2048 keypoints) to extract local features from each tile.
3. **Feature Matching:** using LightGlue to match SuperPoint features between all tile pairs.
4. **RANSAC Affine Verification:** validating matches geometrically
5. **Mask Generation & Merging:** producing masks out of matching tile pairs

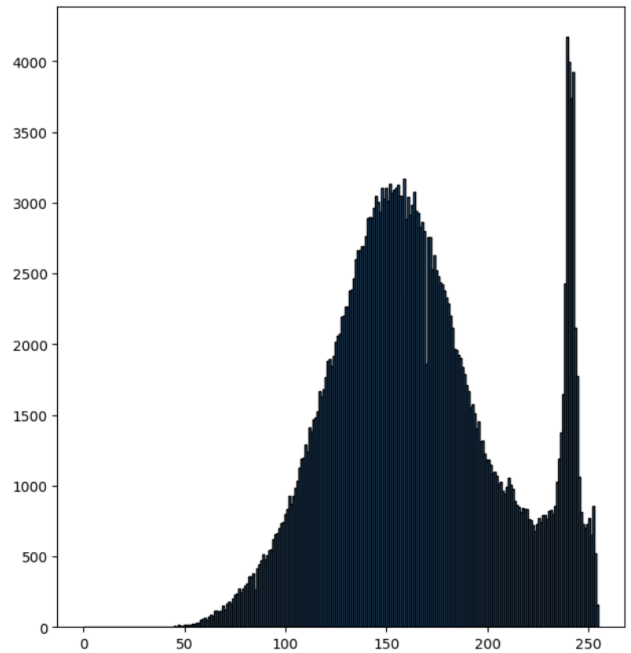
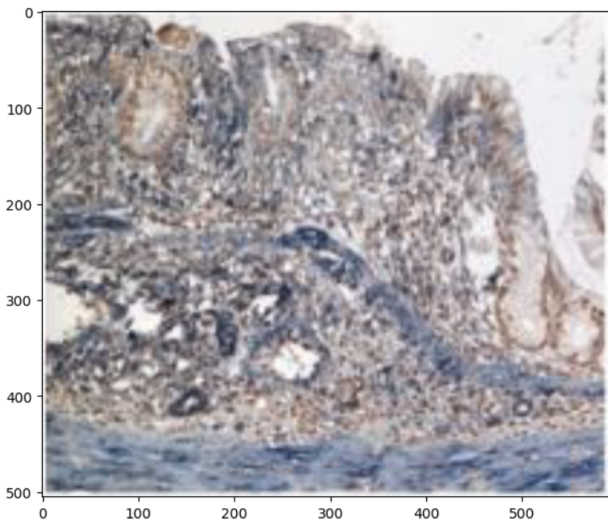
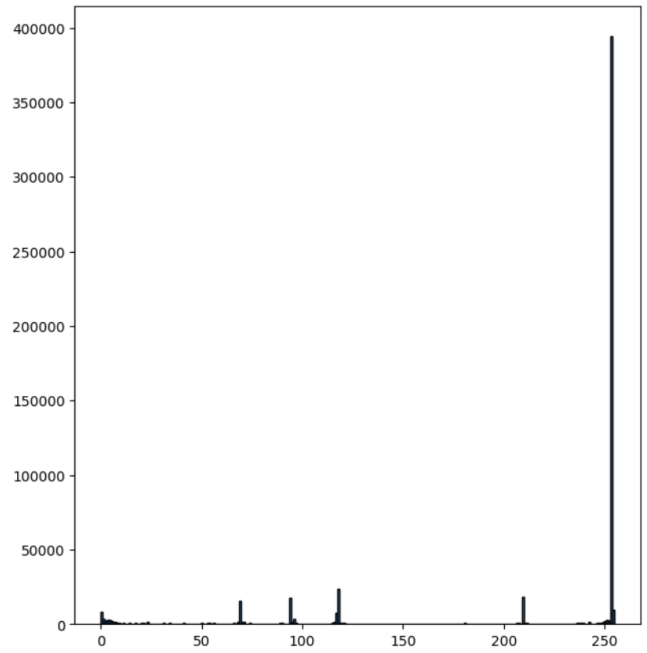
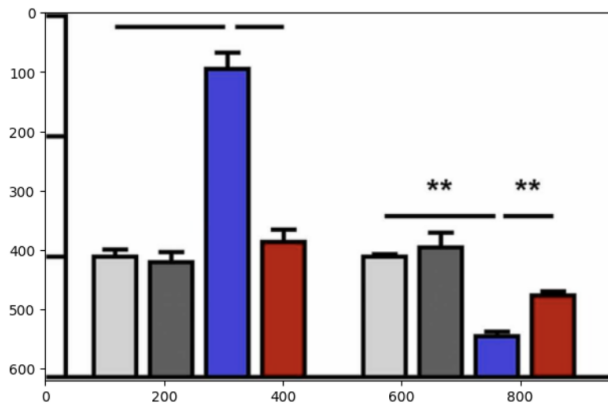
Let me dive into more details for each step.

Tile Detection

1. The image is first binarised with a threshold of 245 over grayscale version of image and then inverted.
2. Morphological open+close 3x3 kernels are run on top to remove white noise on black (open) as well as black noise on white (close).
3. Continuous regions are found with contours, representing objects on the background.
4. Regions are fit into bounding boxes - these are initial candidates for tiles
5. Candidate tiles are then filtered: should be $\geq 0.5\%$ of image size, width and height $\geq 50\text{px}$.

There were two issues with this algorithm:

1. Sometime actual tiles were organised in tables with no white background separators, and thus such tables were detected as one large tile. To deal with such cases I've introduced separate post-processing to try and detect grid lines. To do so I've used Canny edge detection and required $> 50\%$ of pixels be an edge in a given row or column. Upon detecting grid lines, the tile was split into sub-tiles. Unfortunately, while this algo was correctly handling tables, it produced false splits in other cases and dropped the public LB score, despite improving the score on local validation. So it was turned off.
2. Generated images like plots and diagrams were detected as tiles. They contained texts, axes and icons that were detected as matches resulting in false positives regions. To filter out such tiles, I've used heuristics based on grayscale histogram. The idea is simple: plots and diagrams use small amount of colors and thus their histograms will contains sharp spikes, while biological images will have smoother histograms.



Feature Extraction

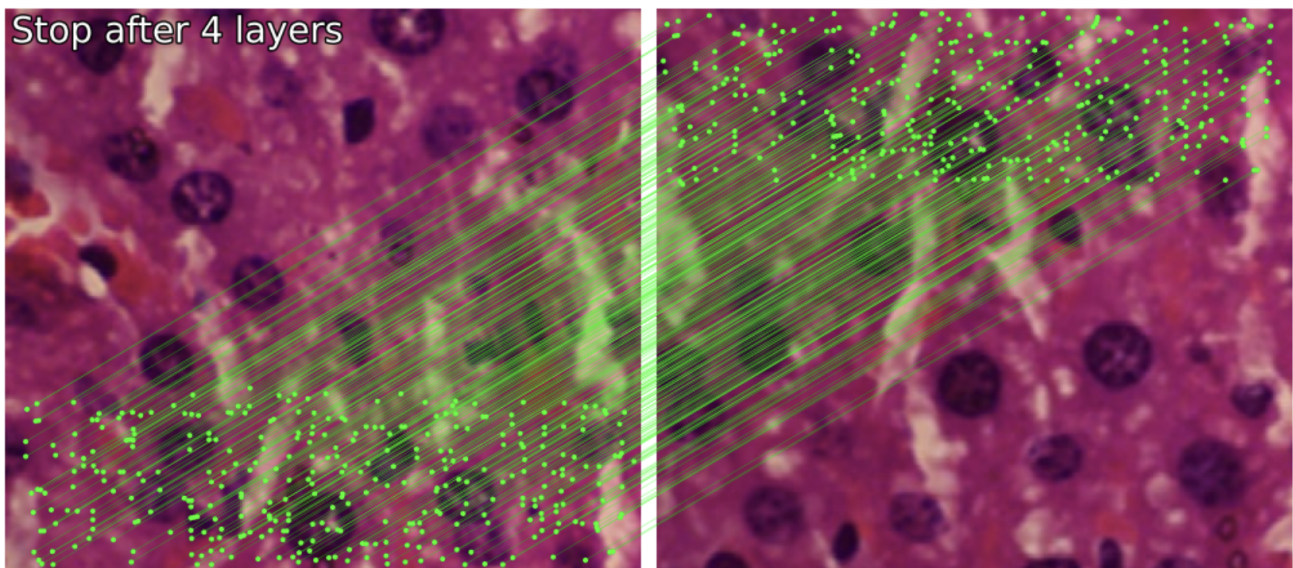
Superpoint was used to detect 2048 most interesting points per tile and extract their features as 256-dim vectors.

Any texts were producing points and resulting in false positives matches. So a separate post-processing was introduced to detect texts and remove any points associated with them. Text was detected with DB50 model.

Feature Matching

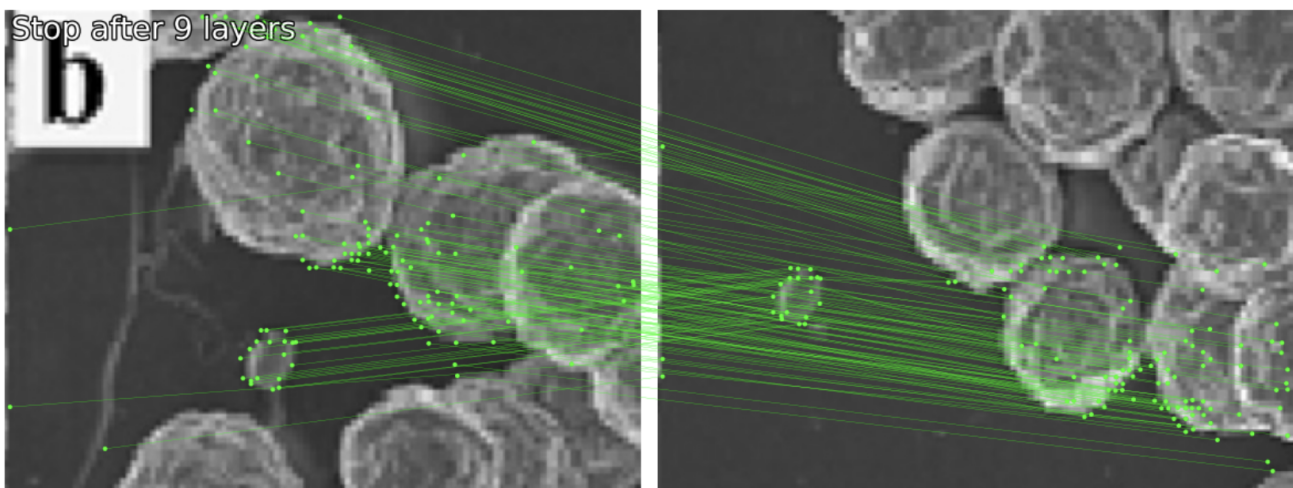
LightGlue was used to match points from one tile with points from another tile. For some tile pairs the LightGlue was stopping matching early due to low match scores, resulting in under-detection. This is why I had

to disable early stop explicitly, increased inference time was not a problem. The points were then filtered by match score threshold, leaving only the pairs of points with decent match confidence.



RANSAC Affine Verification

At this stage there were pairs of matching points potentially containing some noisy matches from different parts of the image. So to further narrow down the matching region, an affine transform was fit to the points, requiring a group of points to behave as a whole: the position of matching points on tile T1 and T2 should be described with the same/similar affine transform. For this RANSAC method from OpenCV library was used, with requirements to have at least 20 inliers and at least 20% inlier ratio among points.

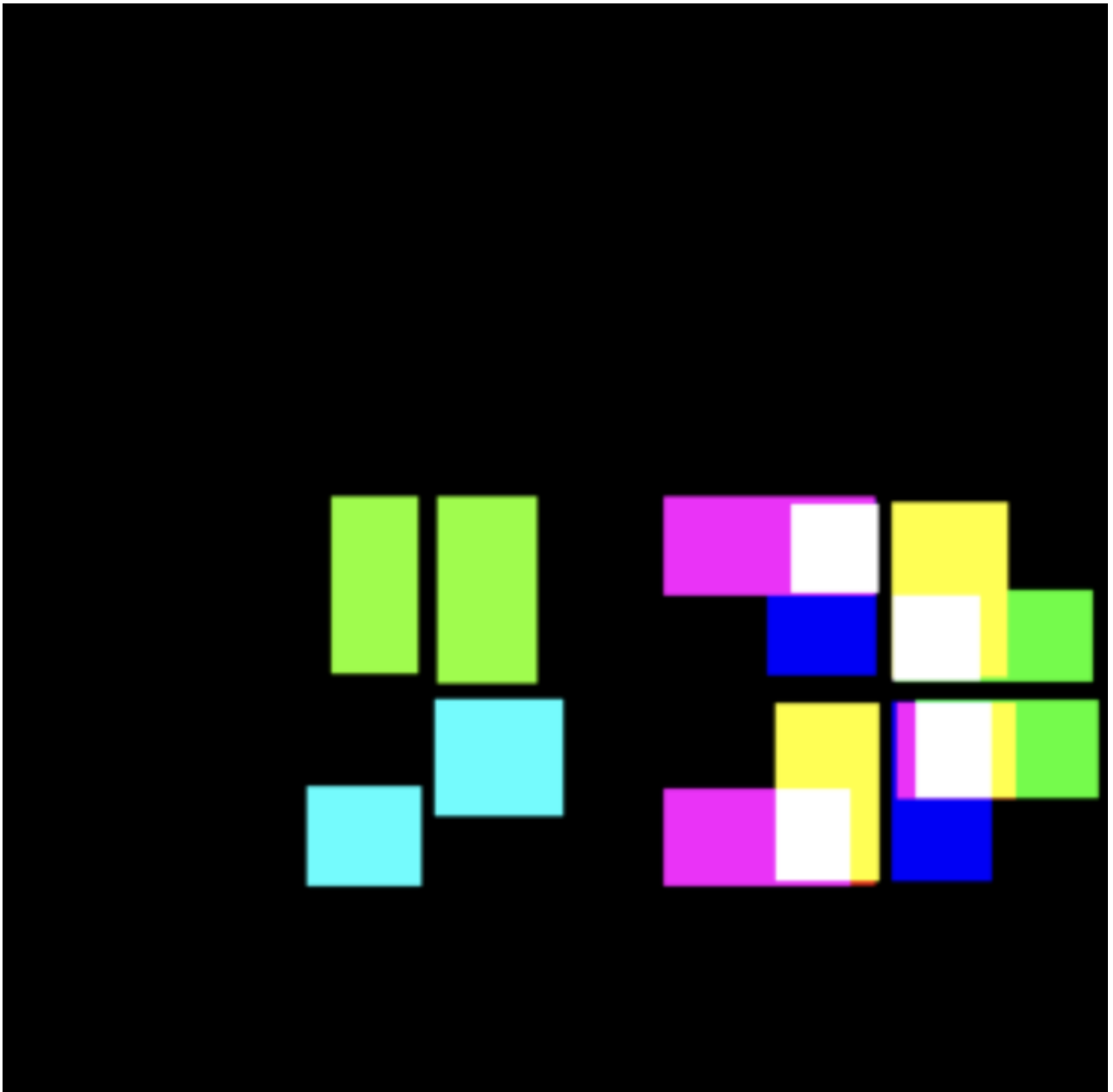


Then minimum area rotated bounding box was computed around the inlier keypoints in each tile, followed by axis-aligned bounding boxes calculation.

Mask Generation & Merging

Bounding boxes for matched regions were converted to masks on the original image.

Sometimes mask were intersecting (for example the matching regions were A - B and A - C), in such case masks were merged into the same layer (requiring intersection-over-union to be at least 0.1).



Additional thoughts

I want to thank the organizers and Kaggle for hosting this competition.

After the submission phase ended, I still had hope for my solution despite being far outside the medal range on the public leaderboard. Unlike most public solutions based on DINO training, I used a point-matching approach, which I expected to be more robust to distribution shifts in new data. Still, finishing in 6th place came as a big surprise to me.